ChatGPT

Inside vLLM: A Beginner's Guide to High-Throughput LLM Inference

Chapter 1: Introduction to Large Language Models and Transformers

Large Language Models (LLMs) are AI systems trained to understand and generate human language. At their core they use the **Transformer** architecture – an artificial neural network based on *multi-head self-attention* [1]. In simple terms, the transformer converts text into a sequence of tokens (word pieces or bytes), looks up each token in an embedding table, and then processes these token vectors through a series of layers. Each layer uses *self-attention* to let every token "pay attention" to (i.e. learn from) all the other tokens in the input. This creates contextualized embeddings: for example, in the sentence "The **bank** will lend me money," the token "bank" learns from its neighbors to understand it refers to a financial institution rather than a riverbank. Finally, a linear layer (often called the output head) computes scores (logits) over the vocabulary to predict the next token.

During inference (text generation), the model is run step by step. First the input prompt is tokenized and fed through all layers (this is called the *prefill* or *prompt encoding*). The model outputs logits for the next token. We sample or choose the next token, append it to the sequence, and run **one more forward pass** (called *decode* or *generation* step) to get the following token, and so on. Importantly, after the prompt is processed once, the model doesn't need to reprocess the entire prefix for each new token; it caches intermediate "key/value" representations from each layer (the KV cache) so that each new token only goes through the model once [2]. This caching is critical for fast generation: without it, generating 100 tokens would require redoing the same computation 100 times; with KV cache, most of that work is done just once [2].

While Transformers are very powerful, they are also resource-intensive. A single large model may have billions of parameters (as in GPT-3 or LLaMA), requiring a lot of memory and compute to run. That's why we use specialized hardware like GPUs and careful engineering. Before delving into vLLM's design, we'll review how GPUs accelerate deep learning and what an "inference engine" does.

Chapter 2: GPU Acceleration for Deep Learning

GPUs (Graphics Processing Units) are the workhorses behind modern deep learning. Unlike a CPU with a few cores, a GPU has thousands of simpler cores optimized for **parallel computation**. In particular, GPUs excel at the matrix-multiply operations at the heart of neural networks. Most neural network layers (including transformer layers) boil down to multiplying large matrices of numbers. GPUs have *CUDA cores* that perform floating-point arithmetic in parallel across many data points. Newer GPUs also have **Tensor Cores** – specialized units that can perform small matrix multiply-accumulate operations (e.g. 4×4 matrix multiplications) in a single cycle [3]. For example, an NVIDIA Volta GPU packs hundreds of Tensor Cores, which together can multiply and accumulate vectors much faster than standard CUDA cores. In practice,

Tensor Cores give 2–12× speedups on the types of half-precision or bfloat16 math used in deep learning [3].

Another key GPU resource is **VRAM** (Video RAM), the onboard high-speed memory. VRAM stores the model weights (potentially gigabytes for large LLMs) and intermediate activations (like the KV cache). If a model or its cache doesn't fit in VRAM, the GPU can't run inference. vLLM carefully checks VRAM usage during setup to make sure the model and its data structures fit within the allowed budget [4]. This is why you often see options like `gpu_memory_utilization=0.8`, meaning "use up to 80% of VRAM" so there's some headroom for other needs.

Launching operations on the GPU (like running a layer) incurs a small overhead from the CPU side. When generating text token by token, this overhead can add up. vLLM uses **CUDA Graphs** to reduce this overhead [5]. CUDA Graphs allow you to record a sequence of GPU operations once and then replay them with one call. vLLM "warms up" by running some dummy passes and capturing them in a graph. Later, each inference step simply replays the pre-recorded graph on the GPU, cutting down the launch overhead and improving latency [5].

In summary, modern GPUs provide massive parallel throughput (via CUDA and Tensor Cores) and high-bandwidth VRAM, both of which are exploited by frameworks like vLLM. Next, we'll see how an inference engine organizes the work of running a model on GPUs to serve many requests efficiently.

Chapter 3: Inference Engines – Synchronous vs. Asynchronous, Latency vs. Throughput

An **inference engine** is the software that takes user requests (text prompts) and runs the model to generate text. Its goals are to maximize throughput (tokens per second) and minimize latency (time to first token and per-token delay). There are two basic modes of serving: *synchronous* and *asynchronous*.

- **Synchronous inference:** The engine receives a fixed batch of requests (prompts) and runs them end-to-end. No new requests can enter mid-way. This is simpler but not ideal for a live server since it can't easily handle newly arriving requests until the current batch is done. The example code below is synchronous: it creates an engine, submits some prompts, generates outputs, and then exits. In this mode, all work is done on a single process/GPU, and execution "blocks" until completion.
- **Asynchronous inference:** The engine continuously accepts new requests even while previous ones are being processed. It typically runs in a loop or event-driven style, repeatedly scheduling any available work. This is important for low-latency serving: after returning an answer for one request, the engine immediately checks if new requests have arrived. The vLLM engine supports fully asynchronous serving, known as *continuous batching*. After each token generation step, it will schedule both old and new requests for the next step [6]. In other words, the engine never idles; it keeps packing any ready prompts into the next GPU run. The figure below sketches this engine loop:

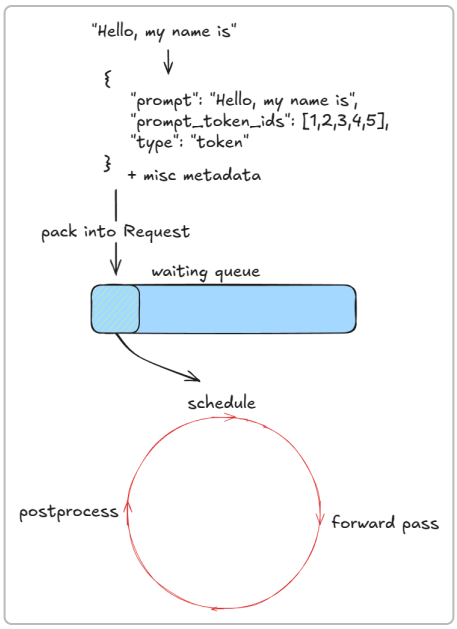

Figure: vLLM engine loop. Each cycle, the engine schedules requests, runs a model forward pass to generate tokens, and then post-processes outputs [7] .

In the figure you see that each "step" has three stages: **Schedule** (decide which requests to process next), **Forward** (run the model on those tokens), and **Postprocess** (sample tokens from the output, append to sequences, and check if any requests are finished). This loop repeats until all requests are complete. Because vLLM flattens all active sequences into one long tensor (a "super-sequence") each forward pass, it can efficiently handle multiple prompts at once, a technique called **continuous batching** [8] [9] . Even in a synchronous setting, this flattening means new prompts can be bundled into the next step if they arrive.

Another important axis is single-GPU vs multi-GPU (and multi-machine) execution. A single-GPU setup is simplest: all model weights and data stay on one GPU. For very large models that exceed one GPU's memory, or for serving extremely high load, vLLM can scale out. It can use PyTorch's tensor parallelism (splitting tensors across GPUs) or pipeline parallelism (staging layers on different GPUs). In practice, vLLM wraps this in a **MultiProcExecutor**: it launches one process per GPU (sometimes with each process handling one shard of the model) and coordinates them via inter-process communication. We'll see more about that later (Chapter 6).

Finally, **latency vs. throughput tradeoffs:** Prefill (running the model on a long prompt) is *compute-bound*, since you run many layers on many tokens. Decoding (one-token generation) is *memory-bound*, as you already have most computations done and just retrieve and multiply a few matrices. vLLM's scheduling strategy prioritizes decode requests, because after a prompt is started, each new token step should be as fast as possible to minimize delay. It also preallocates resources to minimize waiting. Later we'll see techniques (like disaggregated prefill/decode) that further balance latency and throughput [10] [11] . For now, the key point is: **The inference engine must juggle incoming prompts, batch them for GPU runs, and manage memory to deliver high throughput without making any single request wait too long**.

Chapter 4: The Core of vLLM – Architecture and Memory Management

The heart of vLLM is its **engine core**, which orchestrates model execution on the GPU. Its main components are:

- **Scheduler and Queues:** The engine keeps two main queues of requests – *waiting* and *running*. The scheduler repeatedly picks requests from the waiting queue (prefill or decode tasks) and moves them into the running queue for execution. It implements policies (e.g. priority vs FIFO) and ensures resources (like memory) are available before scheduling.
- **KV-Cache Manager (Paged Attention):** This handles memory for storing and retrieving key/value vectors from past tokens. Instead of allocating one giant cache tensor per request, vLLM breaks the KV cache into many fixed-size *blocks*. There is a global *free_block_queue* that holds all available blocks in GPU memory [12]. When a request is scheduled, vLLM allocates as many blocks as needed for its new tokens. Later, when the request finishes or its memory is evicted, its blocks are returned to the free pool. This is analogous to virtual memory paging in an OS: each block is like a page of KV cache [12]. Using blocks (default size 16 tokens for many models) keeps memory use flexible and efficient. For example, the diagram below shows 10 tokens being split into 3 blocks of size 4: these blocks (IDs 1, 2, 3) were taken from the free queue and then will be returned when the request ends.

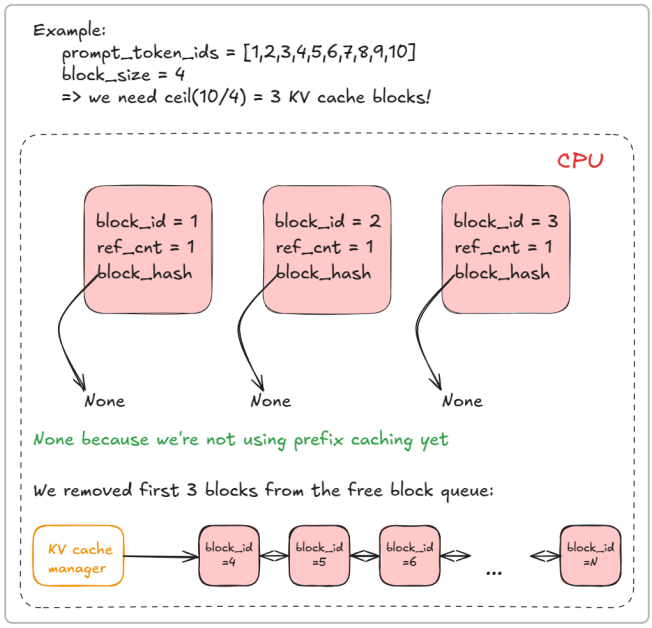

Figure: Example of KV-cache block allocation. Each block is 4 tokens. Tokens 1–10 need 3 blocks; these are removed from the free block queue for use [13].

As shown, the free block queue (in gray) supplies blocks to requests. In paged attention, any request's context tokens are mapped to these blocks. This block-based scheme lets vLLM pack KV memory densely and share it flexibly across many prompts (more on prefix caching soon).

- **GPU Work Execution:** Each GPU worker process (on one card) holds a model runner and the active KV cache blocks. A worker is initialized by *assigning a CUDA device, checking VRAM is available* (given a

utilization target) [4], loading the model weights, and setting up the KV cache. During initialization, vLLM also does some dummy passes and *captures CUDA graphs* for each batch size; these graphs will later be replayed for faster execution [5].

Once running, a step looks like this: all sequences in the running queue are batched together (flattened into a super-sequence) and a single forward pass is run on the model. Because vLLM uses custom kernels that respect the block-indexing, each sequence only attends to its own tokens despite being concatenated. Position indices and masks are constructed so that tokens from different prompts don't mix. After the model forward (which uses *paged attention* to read from only the relevant KV blocks), we gather the output hidden state of the last token of each sequence and compute the logits for the next token. Then we sample tokens (greedy, top-k, top-p, etc.) and append them to each request. If a request hits a stopping condition (max length, end-of-text, or a stop string), vLLM releases its KV blocks back to the free pool.

Because continuous batching is used, after each step both old requests (waiting for more tokens) and any new ones can be scheduled. This is efficient: the diagram below shows an example of how multiple prompts are flattened into one forward pass. Here we have four prompts of different lengths; they are concatenated, and after the forward we extract each final state to sample new tokens. (Notice the KV blocks from previous steps are already in place from memory.)

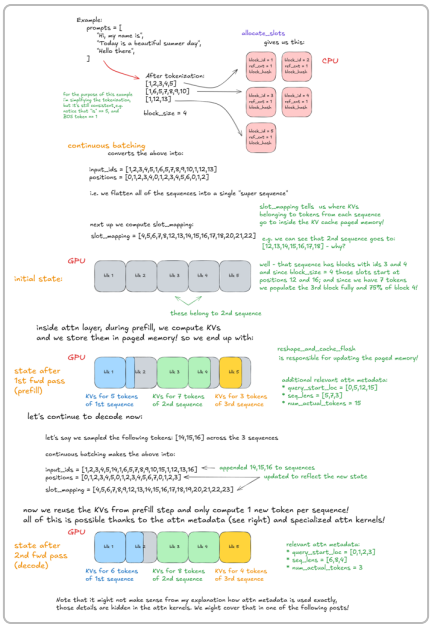

Figure: Continuous batching and paged attention in action. Multiple prompts (A-D) are flattened for a single forward pass. Each prompt's final token output is gathered for sampling. KV-cache blocks (in gray) from previous steps are indexed in memory to provide context [14].

In the figure, the input IDs from all prompts are packed together, and special slot mappings tell the model which tokens belong to which prompt. The attention metadata ensures tokens attend only within their prompt. This lets vLLM run one big matrix multiply instead of many small ones, improving GPU efficiency.

Together, these components allow vLLM to process many prompts on one GPU in an optimized loop. Now let's look at some advanced features built on top of this core logic.

Chapter 5: Advanced Features of vLLM

Chunked Prefill for Long Prompts

Long prompts can be problematic: a single very long request could dominate one engine step, delaying all others. **Chunked prefill** solves this by splitting a long prompt's prefill pass into smaller pieces. In practice, vLLM enforces a cap on how many new prompt tokens can be processed in one step. For example, with a threshold of 8 tokens per step, a 18-token prompt would be handled in 3 chunks (8+8+2). This ensures that after processing 8 tokens, the scheduler can switch to another request if available. The figure below illustrates this: a long request (18 tokens) is broken into chunks across multiple engine steps, so that other prompts get a chance to run in between [15].

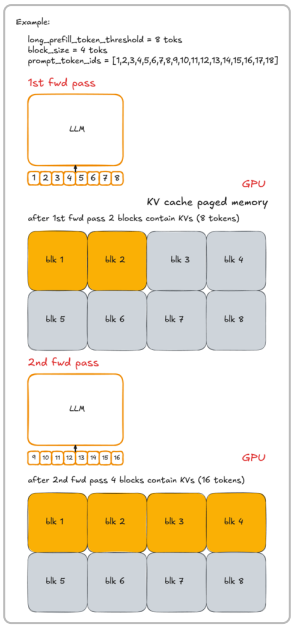

Figure: Chunked prefill example (threshold=8). A long prompt of 18 tokens (`x-y-z`) is split over multiple engine steps, sampling a new token only after all chunks are processed [15].

As shown, the first 8 tokens are processed (step 1), then the next 8 (step 2), then the final 2 (step 3). Only after step 3 do we sample a new token from the distribution. By interleaving steps, we prevent one request from stalling the engine. You can enable this by setting `long_prefill_token_threshold` in vLLM's config.

Prefix Caching

Many use-cases have users sharing a common prefix (for example, "In summary," or a fixed system message in chat). vLLM's **prefix caching** makes use of this: if multiple requests share the same initial tokens, vLLM computes their KV blocks once and reuses them. Practically, vLLM does this by hashing each block of prefix tokens. When a prompt arrives, it splits the shared prefix into 16-token blocks (or whatever the block size is) and computes a hash for each. If that hash exists from a previous request, the engine will reuse the corresponding KV block instead of recomputing it.

Consider this example: we define a long common prefix of length exactly a multiple of the block size, say 32 tokens (two blocks of 16). We then query two different prompts that both start with this prefix. On the **first** request, vLLM will allocate and compute both blocks for the prefix (and store their hashes). On the **second** request, the engine finds both block hashes in its cache. It simply reuses those precomputed blocks from memory. This means for the second prompt, the model only has to compute the **new** tokens after the prefix – the shared part is "free." In effect, the expensive prefix forward pass was done once and its results reused.

The diagrams below illustrate this logic. In step 1, a request with `long_prefix + prompt0` is processed: all prefix blocks are new (hashed and stored), so they are computed normally. In step 2, when `long_prefix + prompt1` arrives, the hashes for the prefix blocks match previously cached blocks. vLLM retrieves those blocks directly from the cache, so only the suffix of prompt1 is processed.

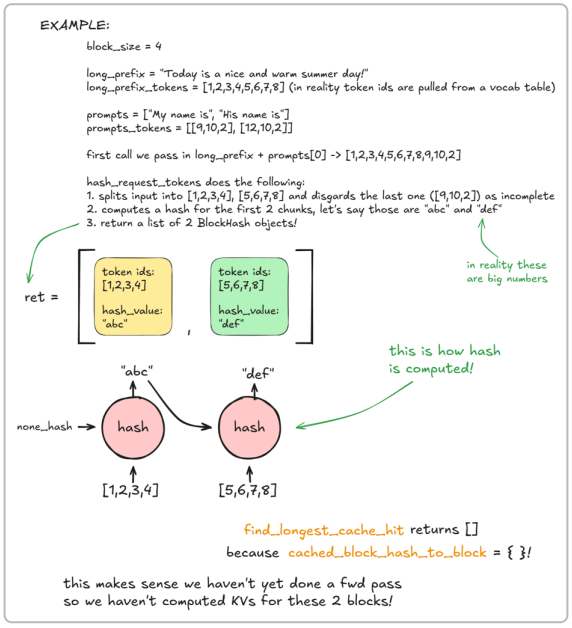

Figure: Prefix caching (part 1). First request with a shared long prefix: blocks are hashed and allocated (no cache hits). The hashed blocks are stored in `cached_block_hash_to_block`.

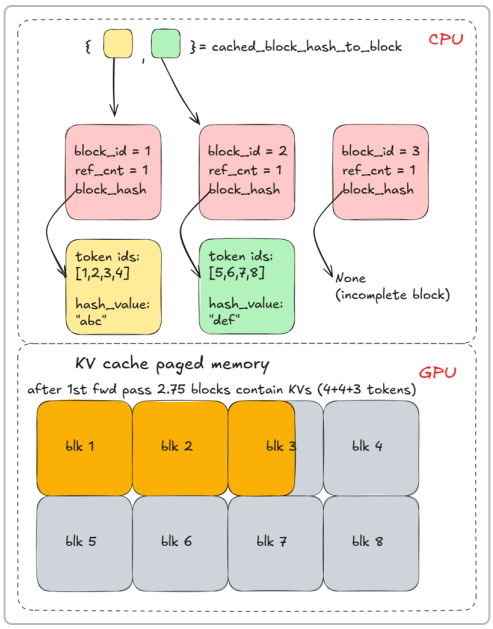

Figure: Prefix caching (part 2). Second request with the same prefix: the engine finds matching hashes and reuses the cached KV blocks from memory, avoiding recomputation.

This prefix caching is on by default. It greatly speeds up cases where many prompts share a beginning. Note that it helps only during prefill (the initial pass). Once decoding (token-by-token generation) starts, prefix caching no longer applies – the cache is for the shared prefix only.

Guided Decoding with Grammars (FSMs)

Sometimes you want to constrain generation to a certain pattern or grammar. vLLM supports **guided decoding** via finite-state machines (FSMs). You can provide a small FSM over tokens (or characters) that describes the valid output sequences. During generation, vLLM will mask out any token that isn't allowed by the current state of the FSM, ensuring the output follows the grammar.

For example, suppose we have an FSM that only allows answers "Positive" or "Negative", character by character. At the first token, only "P" or "N" are allowed; if "P" is chosen, the FSM moves to a state where only "o" is allowed next (to spell "Positive"), and so on. The figure below is a toy example of such an FSM (left) and the corresponding logit-masking process (right). vLLM maintains a bitmask of allowed tokens for the FSM's current state and multiplies logits by this mask (setting disallowed tokens to –∞) [16] . The result is that only grammar-compliant tokens can ever be sampled. This technique lets you enforce anything from simple regex-like patterns to full context-free grammars on the output [17] .

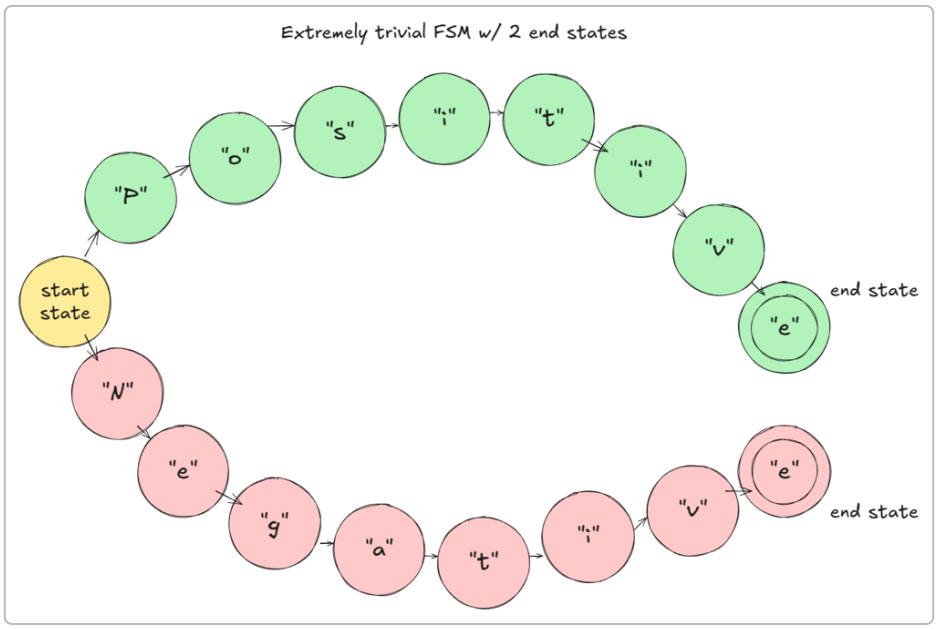

Figure: Example FSM for guided decoding. At each step, the FSM (left) restricts which tokens can be emitted. vLLM masks out disallowed tokens in the logits (right) so only valid transitions occur [17] .

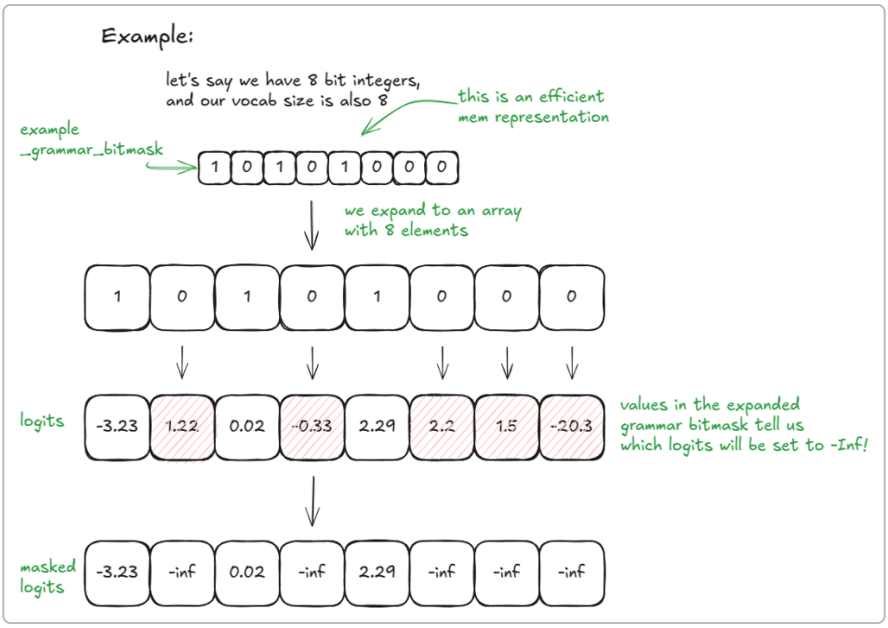

Figure: Logit masking by FSM state. The bitmask (a binary pattern) is expanded to vocab size. Bits set to 1 correspond to allowed tokens; bits 0 are masked to –∞ [16] .

Guided decoding is useful, for example, in programming (forcing syntactically valid code) or structured outputs. It does add overhead (vLLM needs to compute or load the bitmask each step), but it only impacts the sampling step, not the core model forward.

Speculative Decoding

Generating one token at a time with a large LLM can be slow because each token requires a full forward pass. **Speculative decoding** is a speed-up trick: use a small *draft* model (or heuristic) to propose *k* tokens, then verify them with the large model. The algorithm is as follows [18] :

1. **Draft:** Run a small model (or another fast method) on the current context and sample *k* next tokens cheaply.
2. **Verify:** Concatenate the draft tokens to the context and run the **large** model once on this extended sequence (so we get logits for all *k* tokens plus one extra).
3. **Accept/Reject:** Compare the large-model probabilities with the draft's for each of those *k* tokens, left-to-right. If the large model's probability for a draft token is **at least** the draft's probability, we accept it. Otherwise, we accept it with probability `p_large/p_draft` (this rebalances the distribution correctly). We stop at the first rejection (or if all *k* are accepted). If all *k* are accepted, we've effectively generated *k+1* tokens in one large-model pass (the +1 is a final sample from the large model we already computed).

The neat result is that this procedure generates text as if we had sampled one token at a time from the large model, but with fewer large-model passes – potentially batching *k* tokens per pass. In vLLM you can enable speculative decoding with methods like `ngram` (match past contexts) or others. The figure below shows a simplified example: a small model drafts "is Aleksa n…" but the large model wants "is Aleksa **Gordic**", so it rejects after "is Aleksa" and the correct "Gordic" is sampled next [18] .

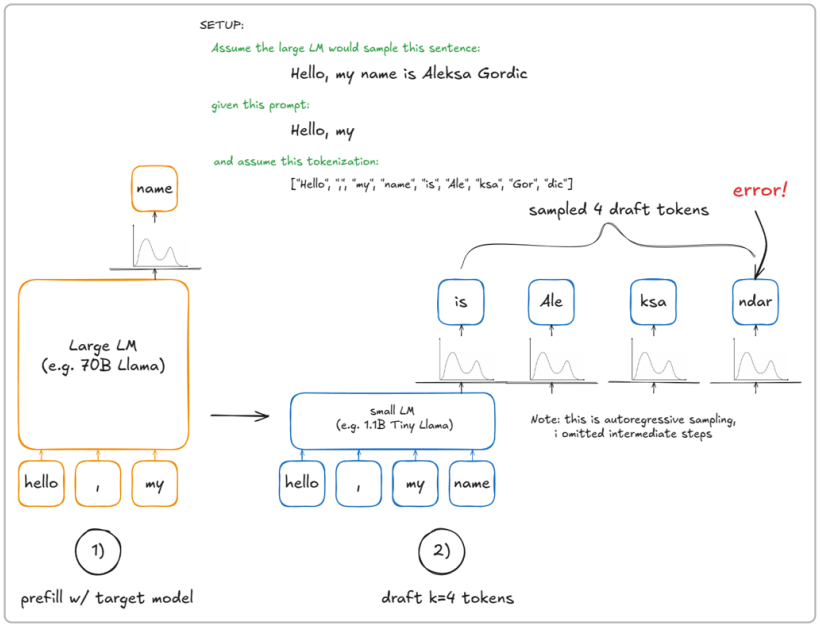

Figure: Speculative decoding example. A small draft model (grey) proposes tokens "is Aleksa ndar". The large model (blue) verifies: "Aleksa" is accepted (because large-model prob ≥ draft), but "ndar" is rejected. We then sample "Gordic" from the reweighted distribution. This way, two tokens ("is", "Aleksa") were emitted with one full-model pass [18] .

Under the hood, vLLM provides `ngram`, `EAGLE`, or `Medusa` draft methods that trade off speed and accuracy. The key takeaway is: **speculative decoding can yield multiple correct tokens per large-model evaluation without changing the end distribution** [18].

Disaggregated Prefill/Decode (Separate Service)

Recall that prefill (long prefix) is compute-heavy and often irregular, while decode (token-by-token generation) is memory-bound and steady. vLLM can **disaggregate** these: run separate worker processes (even on different GPUs or machines) for prefill vs. decode. Prefill workers do the long prompt forward passes and write their KV cache to an external store; decode workers read that cache and continue sampling tokens. This decoupling isolates the bursty prefill from the latency-sensitive decode stage [11].

In practice, you might start one vLLM instance on GPU 0 in "prefill mode" and another on GPU 1 in "decode mode". After the prefill instance finishes its pass (writing KV to a shared connector), it signals the decode instance, which then loads the KV and resumes generation. The diagram below illustrates this disaggregated architecture:

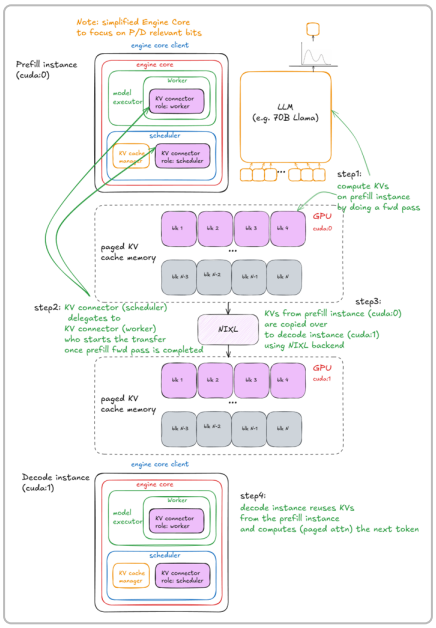

Figure: Disaggregated prefill/decode. Prefill instances handle full prompt passes and upload KV to a shared cache; decode instances fetch KV and emit tokens. This allows independent scaling of compute-bound prefill vs. memory-bound decode, improving overall latency [11].

The code snippet below shows how this might be done in Python using vLLM's `KVTransferConfig` (with a local filesystem connector for simplicity). One process (`run_prefill`) does a prefill with `max_tokens=0` (only sampling 1 token to finalize the prompt), while another (`run_decode`) waits and then generates the rest:

```
from multiprocessing import Process, Event
import os
```

```
from vllm import LLM, SamplingParams
from vllm.config import KVTransferConfig

prompts = ["Hello, my name is", "The president of the United States is"]

def run_prefill(prefill_done):
    os.environ["CUDA_VISIBLE_DEVICES"] = "0"
    sampling_params = SamplingParams(temperature=0, top_p=0.95, max_tokens=1)
    ktc = KVTransferConfig(
        kv_connector="SharedStorageConnector",
        kv_role="kv_both",
        kv_connector_extra_config={"shared_storage_path": "local_storage"},
    )
    llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
kv_transfer_config=ktc)
    llm.generate(prompts, sampling_params)
    prefill_done.set()  # signal that KV cache is ready

def run_decode(prefill_done):
    os.environ["CUDA_VISIBLE_DEVICES"] = "1"
    sampling_params = SamplingParams(temperature=0.8, top_p=0.95)
    ktc = KVTransferConfig(
        kv_connector="SharedStorageConnector",
        kv_role="kv_both",
        kv_connector_extra_config={"shared_storage_path": "local_storage"},
    )
    llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
kv_transfer_config=ktc)
    prefill_done.wait()  # wait until KV is available
    outputs = llm.generate(prompts, sampling_params)
    print(outputs)

if __name__ == "__main__":
    prefill_done = Event()
    Process(target=run_prefill, args=(prefill_done,)).start()
    Process(target=run_decode, args=(prefill_done,)).start()
```

In this example, the `SharedStorageConnector` (using the local filesystem) is used to transfer KV between the two instances. The **prefill** process runs first, writes KV out, and sets an event. The **decode** process then loads the KV into its own memory and continues generation normally. Because prefill and decode run on separate GPUs (or machines), we achieve better resource utilization and lower tail latency [11]. This pattern is especially useful in production systems where prompt lengths and rates are unpredictable.

Chapter 6: Scaling Up – Multi-GPU and Distributed Execution

So far we assumed one GPU per vLLM instance. But large-scale deployments often need multiple GPUs or even multiple nodes. vLLM can scale horizontally using PyTorch's distributed features. One common setup is **Tensor Parallelism (TP)**: the model's weights are split across *N* GPUs, so each layer's computation is done in parallel. vLLM's `MultiProcExecutor` handles this. For example, with TP=4, vLLM will launch 4 worker processes (one per GPU) that together hold the model. These workers communicate through high-speed links (like NVLink).

The figure below shows a MultiProcExecutor with 8-way tensor parallelism. One worker (rank 0) acts as the "driver": it receives requests and broadcasts them to all workers via a shared memory queue. Each worker runs its part of the model in lock-step, and they all share the same KV blocks for their shards. After computation, the driver collects outputs from the designated rank to return to the user. This setup effectively makes *8 GPUs appear as one big GPU* to the rest of the engine, allowing vLLM to serve models up to 8× larger than single-GPU, or to increase throughput proportionally.

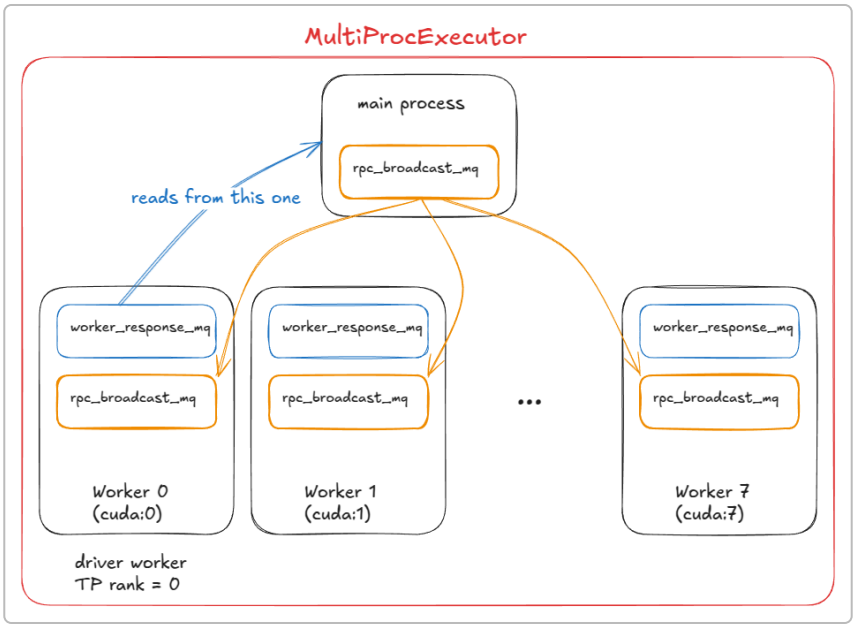

Figure: vLLM MultiProcExecutor with TP=8. One process (rank 0) is the driver; it broadcasts inputs to all 8 GPU workers via a shared queue, and gathers the output from rank 0 after each step. This enables one model to span multiple GPUs [19] .

In practice, you don't usually write multi-GPU code by hand with vLLM. Instead, you configure vLLM with something like `tensor_parallel_size=4`, and it will automatically use the MultiProcExecutor to utilize 4 GPUs. The engine core logic (scheduling, KV cache, etc.) stays the same, just now each forward pass is executed by multiple GPUs in parallel. Tensor parallelism keeps intra-node communication high (fast links), which is why it's preferred over pipeline parallelism if the model fits. For even larger models that exceed even multi-GPU memory, one can use pipeline parallelism (splitting layers across GPUs, possibly across nodes), but that is a more advanced setup beyond this introduction.

Chapter 7: Case Studies and Example Code

Let's look at some concrete ways vLLM might be used. All of the snippets below use the Python API `vllm.LLM` with a `SamplingParams` object.

Basic offline generation: A simple use-case is to load a model and generate text for some prompts. This might run on your laptop or a single server. For example:

```
from vllm import LLM, SamplingParams

prompts = [
    "The capital of France is",
    "Once upon a time in a faraway kingdom,"
]
sampling_params = SamplingParams(temperature=0.7, top_p=0.9)

# Instantiate vLLM (this starts up the engine)
llm = LLM(model="tinyllama-1.1b-chat", dtype="bf16")

# Generate outputs
outputs = llm.generate(prompts, sampling_params)
print(outputs)  # prints the generated continuations
```

This code tokenizes the prompts, does a full prefill for each prompt, and then samples one token (since `max_tokens` defaults to 1 in chat mode unless you change it). The `LLM` object can be reused to generate more tokens iteratively or to handle multiple prompts.

Batching and streaming: In a production server, you might want to handle a stream of prompts, some short and some long. You could run `llm.generate` inside an asynchronous web handler, letting new requests arrive and share work. Thanks to continuous batching, vLLM will automatically group together whatever pending prompts are available on each step. There is no special code you need to write for batching; you just call `generate` possibly in an async loop or on threads, and the engine does the rest.

Multiple GPUs: If you have a bigger model or need more throughput, you can run vLLM on multiple GPUs by configuring tensor parallelism. For example:

```
llm = LLM(model="gpt-3b", tensor_parallel_size=4)
```

This tells vLLM to use 4 GPUs. Internally, it will spawn 4 processes with `MultiProcExecutor`. The rest of your code can remain the same. All scheduling, batching, and caching still happens as before – the only difference is that each forward pass runs on 4 GPUs in parallel.

Integrating with RL or Surrogate Models: Advanced users might combine vLLM with reinforcement learning or other optimization loops. For instance, during preference learning, one can generate answers

with a small lower-level engine (like vLLM) and then score them. The speed of vLLM (with all the optimizations above) makes it easier to do this in real time.

Performance Tuning: In practice, one also monitors throughput and latency. The separate **benchmarks** section of vLLM's docs shows how latency (time to first token and per-token) changes with batch size, GPU type, and engine settings. Users can adjust things like `max_batch_size`, `gpu_memory_utilization`, or enable/disable features like prefix caching depending on their workload.

Overall, vLLM is a flexible library: you mostly write simple code to create the engine and call `generate`, and it handles the rest. The advanced features described above (continuous batching, paged KV, prefix caching, speculative decoding, etc.) are tuned under the hood to give you the best throughput on modern GPUs, with no need for the user to manage them explicitly beyond initial configuration.

Source Links: This guide is based on vLLM's design documentation and related resources [7] [13] [5] [18] [11], with supplementary explanations of transformers and GPU concepts. All quoted material is from the linked sources.

---

[1] Transformer (deep learning) - Wikipedia
https://en.wikipedia.org/wiki/Transformer_(deep_learning)

[2] The Architecture Behind vLLM: How PagedAttention Improves Memory Utilization | by Mandeep Singh | Medium
https://medium.com/@mandeep0405/the-architecture-behind-vllm-how-pagedattention-improves-memory-utilization-2f9b25272110

[3] Tensor (machine learning) - Wikipedia
https://en.wikipedia.org/wiki/Tensor_(machine_learning)

[4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] Inside vLLM: Anatomy of a High-Throughput LLM Inference System - Aleksa Gordić
https://www.aleksagordic.com/blog/vllm